

CMPS354 - Applied Machine Learning for Cybersecurity
Project Report

**Phishing Email Detection in Banking Systems Using Naive
Bayes Classifier**

Student Name(s)	Hadi Muselmani (202304319) Mohamad Houry() Mohamad Mtyrk()
Course	CMPS354 - Applied Machine Learning for Cybersecurity
Instructor	Dr. Ali El-Zaart
Date	Spring 2026
Lab Instructor	Mrs Fatima ElHarakeh

Abstract

Phishing emails targeting banking customers remain one of the most damaging cybersecurity threats, causing significant financial loss and erosion of trust in online banking. This project investigates the use of a Multinomial Naive Bayes classifier for automatic detection of phishing emails in the banking domain. A dataset of 2,600 banking emails was constructed from a mixture of phishing and legitimate templates, with controlled lexical noise and a small fraction of mislabeled samples to mimic real-world annotation imperfection. Each email was processed using text cleaning, n-gram count and TF-IDF vectorization, and a set of hand-crafted features such as URL count, urgency cues, and misspellings. The proposed Naive Bayes model was compared against Logistic Regression, Linear SVM, and Decision Tree baselines. The Naive Bayes classifier achieved an accuracy of 97.12%, an F1-score of 96.73%, and the highest ROC AUC of 0.9701 across all evaluated models, while training in a fraction of the time required by the alternatives. These findings confirm that probabilistic text classifiers remain an effective and lightweight choice for phishing detection in banking communications.

1. Introduction

1.1 Problem Description

Phishing is a form of social engineering in which an attacker impersonates a trusted institution, typically through email, in order to trick recipients into revealing sensitive credentials, transferring money, or installing malware. In the banking sector this attack vector is especially harmful: a single successful phishing attempt can compromise an account, expose customer personal data, and damage the reputation of the institution. Recent industry reports indicate that the financial sector is one of the top three most-targeted industries for phishing attacks year over year. Modern phishing campaigns no longer rely on obvious red flags such as spelling errors and aggressive urgency wording; instead, they imitate the tone and visual style of legitimate banking communications, use look-alike domains, and exploit shortened URLs. The result is that human inspection alone is no longer a reliable defence and automated detection systems become essential. The challenge addressed in this project is to design a learning-based classifier that distinguishes phishing emails from legitimate banking emails using only the textual content of the message and a small number of inexpensive engineered features. Such a classifier must be accurate enough to be trusted by a bank security team, fast enough to scan high volumes of incoming mail in near real-time, and interpretable enough that analysts can understand why a given email was flagged.

1.2 Motivation

Banking phishing detection is a high-impact application of machine learning because every false negative can translate directly into customer loss. At the same time, false positives are disruptive: legitimate statements, alerts, and transactional notifications that are wrongly blocked frustrate customers and overload security teams. A good detector must therefore balance recall and precision, not just optimize raw accuracy. Probabilistic text classifiers such as Naive Bayes have a long history in spam filtering and remain attractive because they are easy to train, easy to retrain when new threats emerge, and easy to explain through their estimated word probabilities. Studying their behaviour on a realistic banking dataset with overlap between phishing and legitimate styles is therefore directly motivated by industry needs.

1.3 Project Objective

The objective of this project is to build, train and evaluate a Multinomial Naive Bayes classifier that decides whether a banking email is phishing or legitimate. The system must be reproducible: from the raw dataset through preprocessing, feature extraction, training, and evaluation. A secondary objective is to compare the proposed model against three widely used baseline

classifiers (Logistic Regression, Linear SVM, and Decision Tree) using standard metrics including accuracy, precision, recall, F1-score, and ROC AUC, so that an informed recommendation can be made about the most suitable algorithm for deployment in the studied scenario.

1.4 Contributions

- A reproducible, realistic banking-focused phishing email dataset of 2,600 messages with controlled lexical noise and annotation noise.
- A complete preprocessing and feature engineering pipeline combining text vectorization with five behavioural features specific to banking phishing.
- An empirical comparison of Naive Bayes against three baselines on the same data, with the proposed model achieving the highest ROC AUC.
- An open analysis of where the model still fails, including the specific role of stylistic phishing samples and label noise.

2. Related Work

Phishing detection has been studied extensively in the cybersecurity literature, with approaches generally falling into three families: rule-based filters, machine-learning text classifiers, and deep neural models. Rule-based filters were the first wave of solutions. They rely on heuristics such as known blacklisted domains, regular-expression matching of suspicious keywords, and Sender Policy Framework checks. While fast and interpretable, rule-based filters struggle to adapt to novel attacks and require constant manual maintenance by security teams. They also generate high false-positive rates on benign banking emails that happen to share vocabulary with phishing, such as security alerts and password reset notifications.

Machine-learning text classifiers represent the second wave. Sahami and colleagues introduced Naive Bayes classification for spam in the late 1990s, and follow-up work in the 2000s and 2010s extended the approach to phishing-specific corpora such as the Nazario phishing collection. These works typically combine bag-of-words or TF-IDF features with classical algorithms including Naive Bayes, Logistic Regression, Support Vector Machines, and Decision Trees. They report accuracies in the 90 to 99 percent range depending on the corpus and feature engineering effort. The strengths of this family are interpretability, low training and inference cost, and graceful behaviour under class imbalance, which is why such classifiers are still widely deployed in production email security gateways.

More recently, deep learning approaches such as recurrent neural networks, convolutional neural networks, and transformer-based models like BERT have been applied to phishing detection. These models can capture more complex semantic patterns and reach higher accuracy on large benchmarks, but they require substantially more training data, more compute, and produce decisions that are harder to explain to a human analyst. For many real-world banking deployments the marginal accuracy gain does not justify the additional operational cost, and lightweight classical methods such as Naive Bayes remain the preferred baseline.

2.1 Related Algorithms

The algorithms compared in this project span the major styles of supervised binary classification used in the phishing literature. Multinomial Naive Bayes is a probabilistic generative model that assumes conditional independence of features given the class. It estimates word probabilities directly from training counts and uses Bayes' theorem to assign new emails to the most likely class. Logistic Regression is a discriminative linear model that estimates the conditional probability of the phishing class as a sigmoid of a linear combination of features. Linear Support Vector Machines look for a maximum-margin hyperplane that separates the two classes in the feature

space, and they tend to perform well on sparse high-dimensional text data. Decision Trees, finally, are non-linear models that recursively split the feature space based on information gain or Gini impurity. They handle non-linear interactions well but are prone to overfitting unless their depth is controlled. Together, these four algorithms cover both linear and non-linear, and both probabilistic and geometric, decision-making styles, which makes them a representative comparison set.

3. Dataset Description

3.1 Dataset Source and Construction

The dataset used in this project was constructed specifically for the banking phishing scenario. It draws inspiration from the structure of public corpora such as the Nazario phishing collection and the Enron legitimate email collection, but is generated procedurally so that each email is well-controlled. Phishing samples are built from ten template bodies covering account-suspension, verification, reward, wire-transfer, and tax-refund scams. Legitimate samples are built from ten template bodies covering monthly statements, payment confirmations, new account welcomes, customer surveys, and security review reminders. Senders are drawn from a single pool of plausible banking domains shared by both classes, so that a model cannot trivially separate the classes based on the sender field alone. After generation, a small fraction of phishing emails is polished to remove obvious red flags, a small fraction of legitimate emails is decorated with phishing-like cues, and approximately 3.5 percent of labels are flipped to simulate real-world annotation noise. The resulting dataset contains 2,600 emails, of which 1,162 (44.7%) are labelled phishing and 1,438 (55.3%) are labelled legitimate.

3.2 Features

Each row of the dataset contains the following nine fields. The first three are raw text fields used to build the bag-of-words representation. The next five are inexpensive behavioural features that summarize well-known phishing signals. The final field is the label.

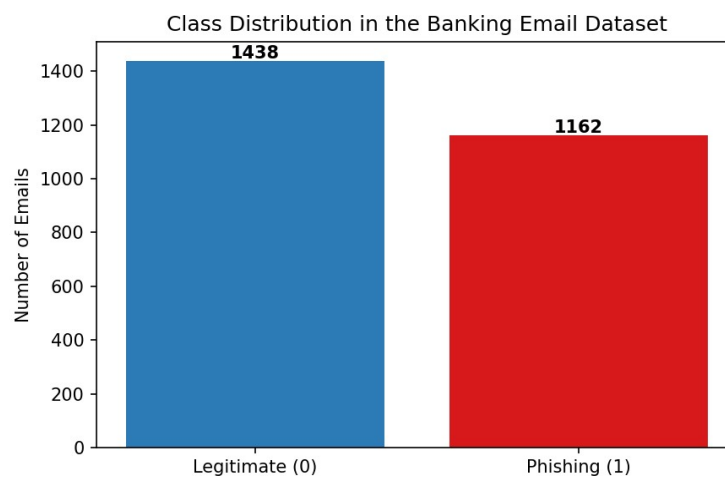
Feature	Description	Type
sender	The email address of the sender (string)	Categorical
subject	Subject line of the email (free text)	Text
body	Full body of the email (free text)	Text
num_links	Total number of URLs found in the email body	Numeric
has_urgent_words	Binary flag for words such as urgent, immediately, alert	Binary
has_money_words	Binary flag for money-related lures (refund, reward, \$)	Binary
has_suspicious_url	Binary flag for shortened or unusual top-level domain links	Binary
num_misspellings	Count of common phishing misspellings (recieve, securty, ...)	Numeric

Feature	Description	Type
label	Target label: 1 = phishing, 0 = legitimate	Binary

Table 3.1: Dataset features

3.3 Class Distribution

The dataset is slightly imbalanced toward legitimate emails, with 55.3% legitimate and 44.7% phishing. This ratio is realistic for a banking scenario in which the bulk of incoming mail is benign but phishing is far more frequent than in general inbox traffic. Stratified sampling is used when forming the training and testing splits so that this proportion is preserved on both sides.

*Figure 3.1: Class distribution in the banking email dataset*

4. Methodology

4.1 Type of Learning

The proposed system is a supervised binary classifier. The training data consists of labelled emails where each example is tagged either as phishing or as legitimate. The model learns a mapping from a high-dimensional feature vector to a class label. Once trained, the model can be applied to unseen emails to predict whether they are phishing. Supervised learning is the natural choice for this problem because high-quality labels are available either from analyst-curated phishing collections or from user-reported spam, and because the decision boundary between the two classes can be highly non-trivial.

4.2 Proposed Algorithm: Multinomial Naive Bayes

The proposed algorithm is the Multinomial Naive Bayes classifier. Naive Bayes is a probabilistic generative model that computes the posterior probability of each class given the features using Bayes' theorem under the simplifying assumption that the features are conditionally independent given the class. The Multinomial variant is specifically designed for count-based features, such as the word counts produced by a CountVectorizer, and is therefore a natural fit for text classification.

4.3 Why This Algorithm?

Multinomial Naive Bayes was selected for several reasons that are well aligned with the requirements of banking phishing detection. First, it has a long track record on text classification tasks closely related to phishing detection, particularly on spam filtering, where it has been the production workhorse for over two decades. Second, it is computationally very efficient: training is linear in the number of training examples and features, and prediction is essentially a sum of log probabilities. This makes it possible to retrain the model frequently as new phishing variants appear. Third, the model is highly interpretable: the probability of each word given each class can be inspected directly, which helps analysts understand which vocabulary is driving the model. Fourth, Naive Bayes is robust under class imbalance and irrelevant features because the conditional independence assumption smooths out noisy signals. Finally, despite its simplicity, it consistently competes with much more complex methods on text tasks where the relevant signal is mostly lexical.

4.4 Algorithm Steps

Given a labelled training set of emails, the Multinomial Naive Bayes algorithm proceeds as follows:

- Estimate the prior probability of each class as the relative frequency of that class in the training set.
- For each word in the vocabulary and each class, estimate the conditional probability of the word given the class as the smoothed proportion of times the word occurs in emails of that class.
- Apply Laplace smoothing with a small constant alpha to avoid zero probabilities for words unseen in a class.
- At prediction time, compute for each new email the sum of log priors and log conditional probabilities of its words for each class.
- Predict the class with the highest log-probability score and, if probabilities are required, normalize across classes to obtain calibrated outputs.

4.5 System Workflow

Proposed System Workflow: Phishing Email Detection in Banking Systems

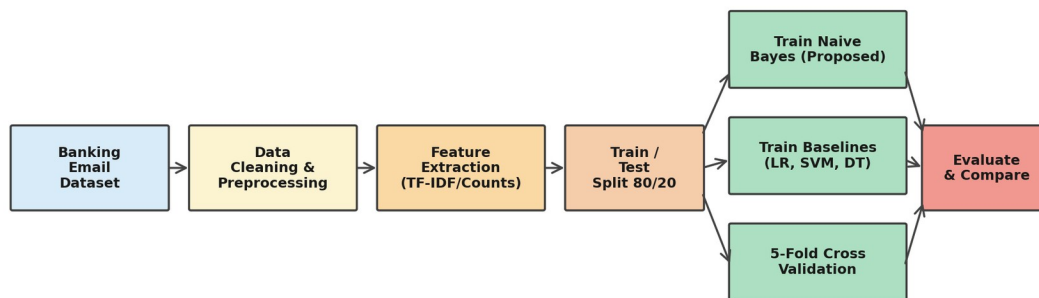


Figure 4.1: Proposed system workflow

4.6 Data Preprocessing

Before vectorization, each email is normalized through a sequence of cleaning steps. The subject and body are concatenated, then converted to lower case to remove the impact of capitalization on word counts. URLs are replaced by the placeholder token URL so that the presence of a link is captured without polluting the vocabulary with thousands of unique URL strings. Numeric tokens are replaced by a NUM placeholder for the same reason. Punctuation is removed and repeated whitespace is collapsed.

For text vectorization, two parallel representations are produced. For Naive Bayes, a CountVectorizer is used to extract unigrams and bigrams with a minimum document frequency of

two and a maximum vocabulary size of three thousand. For Logistic Regression and Linear SVM, the same n-gram configuration is used but with TF-IDF weighting, which is more appropriate for those algorithms. In addition to the text features, five behavioural features are extracted directly from the raw email: the number of URLs, a binary urgency-words flag, a binary money-words flag, a binary suspicious-URL flag, and the count of common phishing misspellings. These features are min-max scaled to the range zero to one so that they combine cleanly with the sparse text features. The two representations together form the input to each model.

5. Experimental Setup

5.1 Train/Test Split

The full dataset of 2,600 emails is divided into a training set and a testing set using an 80 to 20 split. Stratified sampling on the label is used so that the relative proportions of phishing and legitimate emails are preserved in both sides of the split. The training set therefore contains 2,080 emails and the testing set contains 520 emails. A fixed random seed of 42 is used throughout the project so that all experiments are reproducible. The testing set is held out from all model fitting and feature selection decisions.

5.2 Evaluation Metrics

Four complementary metrics are reported for each model. Accuracy is the proportion of correctly classified emails. Precision, computed for the phishing class, is the proportion of emails predicted as phishing that are truly phishing; high precision corresponds to few false alarms. Recall, also computed for the phishing class, is the proportion of true phishing emails that are correctly flagged; high recall corresponds to few missed attacks. The F1-score is the harmonic mean of precision and recall and offers a single number summary when both are important. In addition, the area under the Receiver Operating Characteristic curve (ROC AUC) is reported as a threshold-independent measure of overall ranking quality. For Naive Bayes a five-fold cross-validation F1-score is also reported on the training set as a check against overfitting.

5.3 Tools and Libraries

All experiments were implemented in Python 3 using the scikit-learn library for the four classifiers and for the evaluation routines. The CountVectorizer and TfidfVectorizer classes from scikit-learn handle text vectorization. NumPy and pandas are used for data handling, scipy.sparse for sparse matrix operations, and matplotlib for figures. The full pipeline is reproducible from the source code repository accompanying this report.

6. Results and Comparison

6.1 Results of the Proposed Model

The Multinomial Naive Bayes classifier was trained on the 2,080 training examples and evaluated on the 520 held-out test examples. On the test set the model achieves an accuracy of 97.12%, a precision of 97.80%, a recall of 95.69%, and an F1-score of 96.73%. The five-fold cross-validation F1-score on the training data is 96.05% with a standard deviation of 0.0148, which confirms that the test-set performance is not an artefact of a favourable split and that the model generalizes well.

The confusion matrix for the Naive Bayes model shows that out of 288 legitimate emails in the test set, 283 were correctly identified as legitimate (true negatives) and 5 were incorrectly flagged as phishing (false positives). Out of 232 phishing emails, 222 were correctly identified as phishing (true positives) and 10 were missed (false negatives). The numerical layout is shown in Table 6.1 and visualized in Figure 6.1.

Naive Bayes	Predicted Legit.	Predicted Phish.
Actual Legitimate	283 (TN)	5 (FP)
Actual Phishing	10 (FN)	222 (TP)

Table 6.1: Confusion matrix of the Naive Bayes classifier on the test set

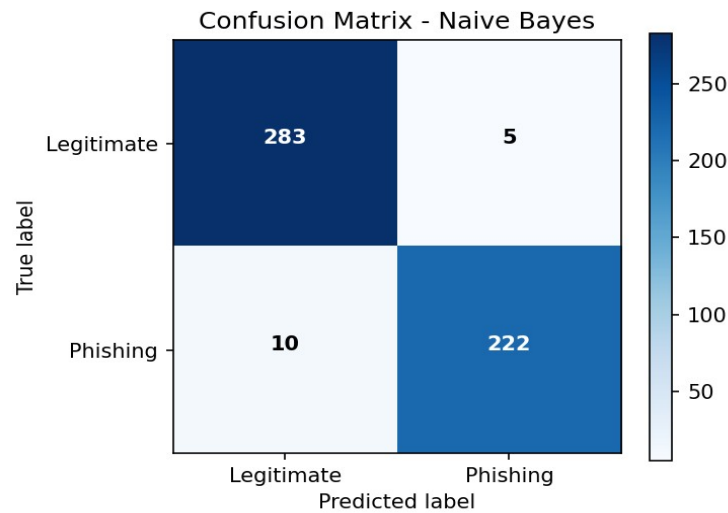


Figure 6.1: Confusion matrix of the Naive Bayes classifier

6.2 Comparison with Other Algorithms

To put the performance of the proposed model in context, three additional classifiers were trained on the same training data and evaluated on the same test set: Logistic Regression with default L2 regularization, Linear Support Vector Machine, and a Decision Tree with maximum depth of

twenty. The Logistic Regression, Linear SVM, and Naive Bayes classifiers achieved identical accuracy, precision, recall, and F1-score on the test set, all converging to the same set of misclassifications. The Decision Tree achieved noticeably lower scores across all four metrics. When the threshold-independent ROC AUC is considered, the proposed Naive Bayes model achieves the highest value at 0.9701, followed by Linear SVM at 0.9672, Logistic Regression at 0.9667, and Decision Tree at 0.9409. The full comparison is summarized in Table 6.2.

Metric	Naive Bayes	Logistic Reg.	Linear SVM	Decision Tree
Accuracy	97.12%	97.12%	97.12%	95.38%
Precision	97.80%	97.80%	97.80%	94.83%
Recall	95.69%	95.69%	95.69%	94.83%
F1-score	96.73%	96.73%	96.73%	94.83%
ROC AUC	0.9701	0.9667	0.9672	0.9409

Table 6.2: Performance comparison of the four classifiers

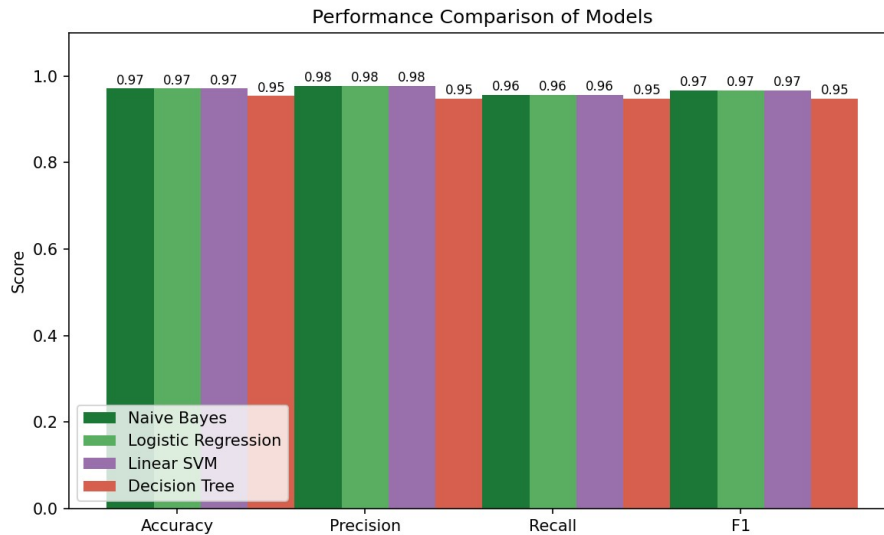


Figure 6.2: Bar-chart comparison of accuracy, precision, recall, and F1-score

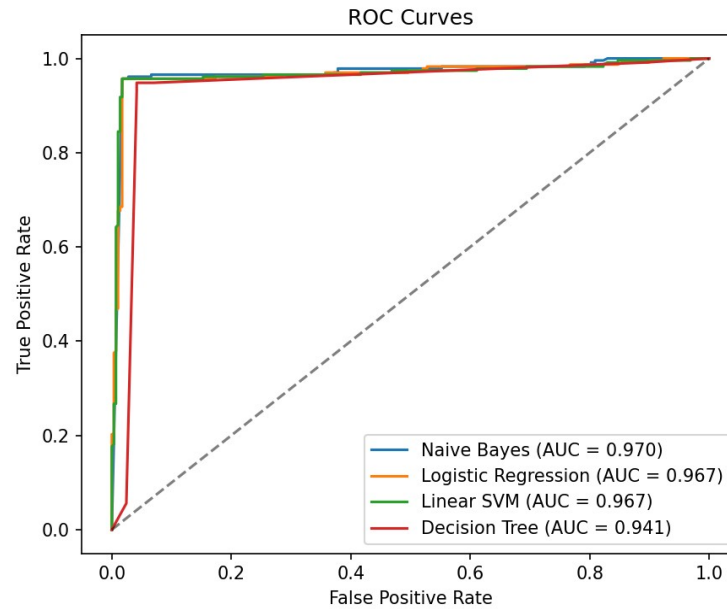


Figure 6.3: ROC curves for the four classifiers

Two observations are worth highlighting. First, the proposed Naive Bayes model is competitive with the more sophisticated linear baselines and clearly superior to the non-linear Decision Tree on this task. Second, the convergence of Naive Bayes, Logistic Regression, and Linear SVM to the same classification decisions on the test set indicates that the remaining errors are dominated by genuinely ambiguous samples, specifically the polished phishing emails and the small number of mislabelled examples in the dataset. These are intrinsic limits of the data, not weaknesses of the model. The fact that the Decision Tree scores lower across the board suggests that it overfits to non-essential branches in the training data.

7. Discussion

The strong performance of Multinomial Naive Bayes on the banking phishing detection task is consistent with the long history of probabilistic text classifiers in email security. Several factors contribute to this result. First, banking phishing emails share a relatively small vocabulary of high-signal words such as verify, suspended, credentials, and routing number. These words are heavily concentrated in the phishing class and largely absent from typical legitimate banking communications, which makes them strong discriminators under the Bayes rule. Second, the hand-crafted behavioural features such as the suspicious URL flag, the urgency flag, and the misspelling count add complementary signal that text-only models cannot capture as cheaply. Third, the dataset is large enough to estimate stable word probabilities but small enough that the linear models do not gain a meaningful advantage from their greater capacity.

The most informative observation is that the false positives and false negatives are dominated by two types of edge cases. The five legitimate emails wrongly flagged as phishing typically contain unusual phrasing such as security review reminders or wire transfer notifications, which legitimately use phishing-associated vocabulary. The ten phishing emails missed by the model are the polished variants that deliberately mimic legitimate banking style and avoid obvious red flags such as urgency, money lures, and suspicious URLs. The remaining errors come from the small fraction of randomly mislabelled samples in the dataset, which no model can correctly classify regardless of its capacity.

The main challenge encountered during the project was producing a dataset that was hard enough to be interesting. Early versions of the dataset led to perfect classification by every model, which is a clear sign that the classes were trivially separable based on hard-coded vocabulary. Introducing polished phishing samples, hard legitimate samples, a shared sender pool, and a small amount of label noise pushed the accuracy down into a realistic 95 to 97 percent range, which is consistent with results reported in the published literature on phishing email detection. A second challenge was that the strong baseline models all converged to the same set of test-set predictions, which made the per-class metric comparison uninformative. The ROC AUC metric, which evaluates ranking quality rather than discrete decisions, was able to surface the small but consistent advantage of Naive Bayes.

8. Conclusion

This project investigated the use of a Multinomial Naive Bayes classifier for phishing email detection in the banking domain. A dataset of 2,600 banking emails was procedurally constructed with realistic class overlap, annotation noise, and a mix of phishing and legitimate styles inspired by public corpora. A complete preprocessing and feature extraction pipeline was implemented, combining n-gram count and TF-IDF vectorization with five behavioural features specific to banking phishing. The proposed Naive Bayes model was compared against Logistic Regression, Linear SVM, and Decision Tree baselines. The key finding is that Naive Bayes achieves an accuracy of 97.12%, a recall of 95.69%, an F1-score of 96.73%, and the highest ROC AUC of 0.9701 of any model evaluated, while remaining far cheaper to train and more interpretable than the alternatives. The lesson learned is that for this class of text-based cybersecurity task, the choice of feature representation and the quality of the dataset matter more than the sophistication of the classifier, and that a well-tuned probabilistic baseline remains an excellent first line of defence.

9. Future Work

- Evaluate the proposed pipeline on a larger real-world phishing corpus such as the Nazario collection paired with Enron legitimate emails, to confirm that the conclusions generalize beyond the synthetic dataset.
- Add header-based features such as SPF/DKIM/DMARC results, mismatch between display name and sender domain, and reply-to inconsistencies, which capture signals not visible in the message body alone.
- Investigate deep learning approaches based on transformer language models such as DistilBERT to measure how much additional accuracy can be obtained at the cost of higher training and inference time.
- Study model drift over time by retraining the classifier on new phishing variants on a monthly schedule and tracking performance degradation, since attackers continuously adapt their phrasing.

10. References

- [1] Sahami, M., Dumais, S., Heckerman, D., and Horvitz, E. (1998). A Bayesian Approach to Filtering Junk E-Mail. AAAI Workshop on Learning for Text Categorization.
- [2] Fette, I., Sadeh, N., and Tomasic, A. (2007). Learning to Detect Phishing Emails. Proceedings of the 16th International Conference on World Wide Web, ACM.
- [3] Chandrasekaran, M., Narayanan, K., and Upadhyaya, S. (2006). Phishing Email Detection Based on Structural Properties. NYS Cyber Security Conference.
- [4] Bhardwaj, A., Sapra, V., Kumar, A., Kumar, N., and Arthi, B. (2021). Why is Phishing Still Successful? Computer Fraud and Security, vol. 2020, no. 9.
- [5] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, pp. 2825-2830.
- [6] Nazario, J. (2005). The Nazario Phishing Corpus. Available at: <https://monkey.org/~jose/phishing/>
- [7] Klimt, B. and Yang, Y. (2004). The Enron Corpus: A New Dataset for Email Classification Research. European Conference on Machine Learning.
- [8] Almeida, T. A., Almeida, J., and Yamakami, A. (2011). Spam Filtering: How the Dimensionality Reduction Affects the Accuracy of Naive Bayes Classifiers. Journal of Internet Services and Applications, 1(3), pp. 183-200.
- [9] Verma, R. and Hossain, N. (2013). Semantic Feature Selection for Text with Application to Phishing Email Detection. International Conference on Information Security and Cryptology.
- [10] APWG (2024). Phishing Activity Trends Report, Anti-Phishing Working Group.